

SERVICIO NACIONAL DE APRENDIZAJE — SENA			
INSTRUCTIVO TÉCNICO PARA APRENDICES			
Programa de Formación:	Análisis y Desarrollo de Software (ADSO)	Código:	228118 v1
Centro de Formación:	Centro de la Industria, la Empresa y los Servicios — Yamboró	Regional:	Huila
Nombre del Instructivo:	Maquetación web con CSS Flexbox: dominio completo del modelo de caja flexible		
Instructor:	Diego Fernando Calderón Silva	Duración estimada:	10 horas
Entrega del proyecto:	21 de junio de 2026 (repositorio en GitHub)	Modalidad:	Individual

DOMINANDO CSS FLEXBOX

El modelo de caja flexible para maquetación web moderna

HTML • CSS • JavaScript

Propósito del instructivo

Flexbox es, hoy, la herramienta fundamental para alinear y distribuir elementos en una interfaz web. Antes de Flexbox, centrar un elemento vertical y horizontalmente o crear una barra de navegación adaptable exigía trucos complejos con floats, posiciones absolutas y márgenes calculados a mano. Flexbox resolvió todo eso de forma elegante. Este instructivo cubre, una por una, TODAS las propiedades del modelo Flexbox —tanto las del contenedor como las de los ítems— con ejemplos de código verificables, y culmina con un proyecto integrador que el aprendiz publicará en GitHub a más tardar el 21 de junio de 2026. Al finalizar, el aprendiz será capaz de maquetar cualquier componente de interfaz sin recurrir a técnicas obsoletas.

Elaborado por el instructor Diego Fernando Calderón Silva

1. Introducción

El módulo de maquetación con caja flexible (Flexible Box Layout, abreviado Flexbox) ofrece una manera eficiente de organizar, alinear y distribuir el espacio entre los elementos de un contenedor, incluso cuando el tamaño de esos elementos es desconocido o dinámico (de ahí la palabra «flexible»).

La idea central detrás de Flexbox es darle al contenedor la capacidad de modificar el ancho, el alto y el orden de sus elementos para aprovechar mejor el espacio disponible, adaptándose a distintos dispositivos y tamaños de pantalla. Un contenedor flexible puede expandir los elementos para llenar el espacio libre, o encogerlos para evitar que se desborden.

A diferencia de los esquemas tradicionales (bloque, que es vertical, e inline, que es horizontal), Flexbox es independiente de la dirección. Esto lo hace ideal para los componentes de una aplicación y para diseños de pequeña y mediana escala. Para diseños de página de gran escala, lo complementa CSS Grid, pero ese es tema de otro instructivo.

2. Objetivos de aprendizaje

Al finalizar este instructivo, el aprendiz estará en capacidad de:

- Comprender la terminología de Flexbox: eje principal, eje transversal, contenedor e ítems flexibles.
- Aplicar correctamente todas las propiedades del contenedor flexible: display, flex-direction, flex-wrap, flex-flow, justify-content, align-items, align-content y gap.
- Aplicar todas las propiedades de los ítems flexibles: order, flex-grow, flex-shrink, flex-basis, flex y align-self.
- Resolver problemas clásicos de maquetación: centrado perfecto, barras de navegación, galerías y layouts adaptables.
- Construir un diseño responsive con Flexbox y media queries, sin floats ni posicionamiento absoluto.
- Versionar un proyecto con Git y publicarlo en un repositorio de GitHub.

3. Requisitos previos

- Conocimientos sólidos de HTML5 (etiquetas estructurales: header, nav, main, section, footer).
- Conocimientos de CSS3 (selectores, modelo de caja, unidades rem/px/%, colores).
- Visual Studio Code instalado, con la extensión Live Server.
- Git instalado y una cuenta activa en GitHub.
- Un navegador moderno con herramientas de desarrollo (Chrome o Edge recomendados).

4. Terminología fundamental

Flexbox no es una sola propiedad, sino un módulo completo con un conjunto de propiedades. Algunas se aplican al elemento padre (llamado contenedor flexible) y otras a los elementos hijos (llamados ítems flexibles). Antes de escribir código, es imprescindible dominar el vocabulario.

4.1 Los dos ejes

Mientras la maquetación tradicional se basa en direcciones de bloque e inline, Flexbox se basa en direcciones de flujo flexible apoyadas en dos ejes:

Término	Significado
Eje principal (main axis)	Eje primario a lo largo del cual se distribuyen los ítems. CUIDADO: no es necesariamente horizontal; su orientación depende de la propiedad flex-direction.
main-start / main-end	Los ítems se colocan desde el inicio (main-start) hacia el final (main-end) del eje principal.
main size	El ancho o el alto del ítem, según cuál esté en la dimensión principal.
Eje transversal (cross axis)	Eje perpendicular al eje principal. Su dirección depende de la del eje principal.
cross-start / cross-end	Las líneas se llenan de ítems y se colocan desde cross-start hacia cross-end.
cross size	El ancho o el alto del ítem, según cuál esté en la dimensión transversal.

La clave que evita el 90% de la confusión

El eje principal NO siempre es horizontal. Si flex-direction es row (valor por defecto), el eje principal es horizontal y justify-content mueve los ítems de izquierda a derecha. Pero si flex-direction es column, el eje principal pasa a ser vertical, y entonces justify-content los mueve de arriba a abajo. Cada vez que use Flexbox, pregúntese primero: ¿en qué dirección está mi eje principal?

5. Propiedades del contenedor (flex container)

Estas propiedades se aplican al elemento padre. Activan el contexto flexible y gobiernan cómo se comportan todos sus hijos directos.

display

Define un contenedor flexible y activa el contexto Flexbox para todos sus hijos directos.

```
.container {  
  display: flex; /* o inline-flex */  
}
```

Con flex, el contenedor se comporta como un bloque. Con inline-flex, se comporta como un elemento en línea. Nota: las columnas CSS (columns) no tienen efecto sobre un contenedor flexible.

flex-direction

Establece el eje principal y, por tanto, la dirección en que se colocan los ítems. Flexbox es (salvo por el ajuste de línea) un concepto de diseño unidireccional: los ítems se disponen o en filas horizontales o en columnas verticales.

```
.container {  
  flex-direction: row | row-reverse | column | column-reverse;  
}
```

Valor	Descripción
row (defecto)	De izquierda a derecha (en escritura ltr).
row-reverse	De derecha a izquierda (en escritura ltr).
column	Igual que row, pero de arriba hacia abajo.
column-reverse	Igual que row-reverse, pero de abajo hacia arriba.

flex-wrap

Por defecto, los ítems intentan caber todos en una sola línea. Con esta propiedad se permite que pasen a varias líneas cuando sea necesario.

```
.container {  
  flex-wrap: nowrap | wrap | wrap-reverse;  
}
```

Valor	Descripción
nowrap (defecto)	Todos los ítems en una sola línea (se encogen si es preciso).
wrap	Los ítems se ajustan en varias líneas, de arriba hacia abajo.
wrap-reverse	Los ítems se ajustan en varias líneas, de abajo hacia arriba.

flex-flow

Es la forma abreviada de flex-direction y flex-wrap combinadas, que juntas definen los ejes principal y transversal. El valor por defecto es row nowrap.

```
.container {  
  flex-flow: column wrap;  
}
```

justify-content

Define la alineación a lo largo del eje principal. Ayuda a distribuir el espacio libre sobrante cuando los ítems son inflexibles o ya alcanzaron su tamaño máximo.

```
.container {
  justify-content: flex-start | flex-end | center |
                 space-between | space-around | space-evenly;
}
```

Valor	Descripción
flex-start (defecto)	Los ítems se agrupan al inicio del eje principal.
flex-end	Los ítems se agrupan al final del eje principal.
center	Los ítems se centran en la línea.
space-between	Distribución uniforme; el primer ítem pegado al inicio y el último al final.
space-around	Espacio igual alrededor de cada ítem. Visualmente los bordes parecen tener la mitad de espacio, porque cada ítem aporta su propio margen.
space-evenly	El espacio entre ítems y hacia los bordes es exactamente igual.

Valores más seguros

El soporte de algunos valores tiene matices entre navegadores. Los tres valores más seguros y universalmente soportados son flex-start, flex-end y center. Los de distribución de espacio (space-between, space-around, space-evenly) funcionan perfectamente en todos los navegadores modernos.

align-items

Define cómo se alinean los ítems a lo largo del eje transversal en la línea actual. Es el equivalente a justify-content, pero para el eje perpendicular.

```
.container {
  align-items: stretch | flex-start | flex-end | center | baseline;
}
```

Valor	Descripción
stretch (defecto)	Los ítems se estiran para llenar el contenedor (respetando min/max-width).
flex-start	Los ítems se colocan al inicio del eje transversal.
flex-end	Los ítems se colocan al final del eje transversal.
center	Los ítems se centran en el eje transversal.
baseline	Los ítems se alinean según la línea base de su texto.

align-content

Alinea las LÍNEAS del contenedor cuando hay espacio extra en el eje transversal, de forma análoga a como justify-content alinea ítems en el eje principal.

⚠ Solo funciona con varias líneas

align-content solo tiene efecto en contenedores multilínea, es decir, cuando flex-wrap está en wrap o wrap-reverse. En un contenedor de una sola línea (nowrap, el valor por defecto) esta propiedad no hace absolutamente nada. Este es un error muy frecuente: aplicar align-content y no ver ningún cambio porque falta el flex-wrap: wrap.

```
.container {  
  flex-wrap: wrap;  
  align-content: flex-start | flex-end | center |  
                space-between | space-around | stretch;  
}
```

gap, row-gap, column-gap

Controla de forma explícita el espacio entre los ítems. Aplica ese espacio solo ENTRE ítems, nunca en los bordes externos del contenedor. Es la forma moderna y limpia de separar elementos, sin recurrir a márgenes.

```
.container {  
  display: flex;  
  gap: 10px;           /* mismo espacio en filas y columnas */  
  gap: 10px 20px;      /* fila 10px, columna 20px */  
  row-gap: 10px;  
  column-gap: 20px;  
}
```

gap reemplaza muchos trucos viejos

Antes se usaban márgenes y selectores como :not(:last-child) para separar elementos. Con gap, el navegador calcula y reserva el espacio automáticamente, sin añadir relleno extra en los extremos y sin riesgo de desbordes. Además, gap también funciona en CSS Grid y en multi-columnas.

6. Propiedades de los ítems (flex items)

Estas propiedades se aplican a los hijos directos del contenedor flexible y permiten controlar el comportamiento individual de cada uno.

order

Por defecto, los ítems se muestran en el orden en que aparecen en el HTML. La propiedad `order` controla el orden visual en que se presentan, sin tocar el HTML.

```
.item {  
  order: 5; /* el valor por defecto es 0 */  
}
```

Los ítems con el mismo valor de `order` se ordenan según su posición en el código fuente. Los valores pueden ser negativos.

⚠ Nota de accesibilidad

Las tecnologías de asistencia, como los lectores de pantalla, anuncian los ítems en su orden original del HTML, no en el orden visual definido por `order`. La reordenación es visual, no estructural. Por eso, cuando el orden de lectura sea importante, conviene reordenar en el HTML y no solo con CSS.

flex-grow

Define la capacidad de un ítem para CRECER si es necesario. Acepta un valor sin unidades que actúa como proporción: indica qué porción del espacio libre del contenedor debe ocupar el ítem.

```
.item {  
  flex-grow: 4; /* por defecto 0 */  
}
```

Si todos los ítems tienen `flex-grow` en 1, el espacio restante se reparte por igual entre todos. Si uno tiene valor 2, ese ítem intentará ocupar el doble de espacio que los demás. No se admiten valores negativos.

flex-shrink

Define la capacidad de un ítem para ENCOGERSE si es necesario, cuando no hay suficiente espacio en el contenedor.

```
.item {  
  flex-shrink: 3; /* por defecto 1 */  
}
```

Tampoco admite valores negativos. Un valor de 0 evita que el ítem se encoja.

flex-basis

Define el tamaño por defecto de un elemento ANTES de que se distribuya el espacio restante. Puede ser una longitud (20%, 5rem, 200px) o la palabra clave auto.

```
.item {  
  flex-basis: 200px | auto; /* por defecto auto */  
}
```

El valor auto significa «mira mi propiedad width o height». Si se pone en 0, el espacio alrededor del contenido no se tiene en cuenta para el reparto.

flex (forma abreviada)

Es la forma abreviada de flex-grow, flex-shrink y flex-basis combinadas. El segundo y el tercer parámetro son opcionales. El valor por defecto es 0 1 auto.

```
.item {  
  /* grow | shrink | basis */  
  flex: 1 1 auto;  
  
  /* atajo común: flex: 1; equivale a flex: 1 1 0%; */  
  flex: 1;  
}
```

Recomendación oficial

Se recomienda usar la forma abreviada flex en lugar de fijar las tres propiedades por separado, porque establece los valores de forma inteligente. El atajo flex: 1 es uno de los más útiles del lenguaje: hace que un ítem ocupe todo el espacio disponible, ideal para que un footer quede pegado al fondo o para repartir columnas equitativamente.

align-self

Permite sobrescribir la alineación definida por align-items, pero para un ítem individual. Acepta los mismos valores que align-items.

```
.item {  
  align-self: auto | flex-start | flex-end |  
             center | baseline | stretch;  
}
```

Nota: las propiedades float, clear y vertical-align no tienen ningún efecto sobre un ítem flexible.

7. Casos prácticos resueltos

Aplicar la teoría es la única forma de fijarla. Estos son los patrones que el aprendiz usará a diario.

7.1 El centrado perfecto

El problema más clásico de CSS: centrar un elemento horizontal y verticalmente. Con Flexbox es trivial.

```
.parent {  
  display: flex;  
  justify-content: center; /* centra en el eje principal */  
  align-items: center;    /* centra en el eje transversal */  
  height: 300px;  
}
```

Existe incluso un truco aún más corto, aprovechando que un margen auto dentro de un contenedor flexible absorbe el espacio sobrante:

```
.parent {  
  display: flex;  
  height: 300px;  
}  
.child {  
  margin: auto; /* ¡magia! centra en ambos ejes */  
}
```

7.2 Barra de navegación con espaciado

Una navegación horizontal de enlaces, separados de forma uniforme, sin desbordes en los bordes:

```
.nav {  
  display: flex;  
  gap: 1.5rem; /* separación entre enlaces */  
  justify-content: space-around;  
}
```

7.3 Galería adaptable sin media queries

Seis tarjetas que se reacomodan solas al cambiar el ancho de la ventana, gracias a flex-wrap y flex-grow:

```
.galeria {  
  display: flex;  
  flex-flow: row wrap; /* permite varias líneas */  
  gap: 1rem;  
  justify-content: space-around;  
}  
.galeria .tarjeta {
```

```
flex: 1 1 200px;      /* crece, se encoge, base de 200px */
}
```

La declaración `flex: 1 1 200px` es la receta mágica: cada tarjeta parte de 200px, pero crece para llenar el espacio y se reacomoda en filas según quepa. Esto se adapta solo, sin una sola media query.

7.4 Layout responsive de página completa

Un diseño de tres columnas con cabecera y pie a todo el ancho, que en móvil se apila en una sola columna:

```
.wrapper {
  display: flex;
  flex-flow: row wrap;
}
/* En móvil: todo ocupa el 100% del ancho */
.wrapper > * {
  flex: 1 100%;
}

/* Pantallas medianas en adelante */
@media all and (min-width: 600px) {
  .aside { flex: 1 0 0; }
}

/* Pantallas grandes: 3 columnas, el centro el doble de ancho */
@media all and (min-width: 800px) {
  .main    { flex: 3 0px; order: 2; }
  .aside-1 { order: 1; }
  .aside-2 { order: 3; }
  .footer  { order: 4; }
}
```

8. Proyecto entregable: «Mi Portafolio Flex»

Fecha límite de entrega

El proyecto debe estar publicado en un repositorio público de GitHub a más tardar el día 21 de junio de 2026. Se evaluará el contenido del repositorio a esa fecha, incluyendo el historial de commits.

8.1 Descripción

El aprendiz construirá una página web de portafolio personal de una sola página (one-page), maquetada ENTERAMENTE con Flexbox. No se permite el uso de floats, position: absolute para maquetar, ni frameworks de CSS (Bootstrap, Tailwind, etc.). El objetivo es demostrar dominio puro del modelo flexible.

8.2 Secciones obligatorias

La página debe contener, como mínimo, las siguientes secciones, cada una maquetada con Flexbox:

1. Barra de navegación (header con nav): logo a la izquierda y enlaces a la derecha, usando justify-content: space-between. Debe colapsar a columna en móvil.
2. Sección de presentación (hero): foto o avatar y un texto de bienvenida, centrados perfectamente con align-items y justify-content.
3. Sección «Sobre mí»: dos columnas (texto e información) que se apilan en móvil mediante flex-wrap.
4. Galería de proyectos/habilidades: mínimo 6 tarjetas en una cuadrícula flexible con flex-wrap: wrap y gap, que se reacomoden según el ancho.
5. Pie de página (footer): enlaces a redes y créditos, distribuidos con Flexbox.

8.3 Requisitos técnicos

- **Uso obligatorio de al menos:** display: flex, flex-direction, justify-content, align-items, flex-wrap, gap, flex (en ítems) y order.
- Diseño responsive con al menos dos media queries (un punto de quiebre para tablet y otro para móvil).
- Uso de etiquetas HTML semánticas (header, nav, main, section, footer).
- Un archivo JavaScript que añada algo de interactividad simple: por ejemplo, un botón de menú hamburguesa que muestre u oculte la navegación en móvil, o un botón que cambie entre tema claro y oscuro.
- Código ordenado, indentado y comentado en las secciones clave del CSS.

8.4 Estructura de archivos sugerida

```
mi-portafolio-flex/  
├── index.html  
├── css/  
│   └── estilos.css  
├── js/  
│   └── app.js  
├── img/  
│   └── (imágenes del portafolio)  
└── README.md    (descripción del proyecto)
```

8.5 Punto de partida (HTML base)

El aprendiz puede comenzar a partir de este esqueleto, que ya define la estructura semántica. Su tarea es maquetarlo con Flexbox y darle vida:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mi Portafolio</title>
  <link rel="stylesheet" href="css/estilos.css">
</head>
<body>

  <header class="header">
    <div class="logo">Mi Nombre</div>
    <nav class="nav" id="menu">
      <a href="#sobre-mi">Sobre mí</a>
      <a href="#proyectos">Proyectos</a>
      <a href="#contacto">Contacto</a>
    </nav>
    <button class="menu-btn" id="menuBtn">☰</button>
  </header>

  <section class="hero">
    
    <div class="hero-texto">
      <h1>Hola, soy [Nombre]</h1>
      <p>Aprendiz de Análisis y Desarrollo de Software</p>
    </div>
  </section>

  <section id="sobre-mi" class="sobre-mi">
    <div class="columna">
      <h2>Sobre mí</h2>
      <p>Una breve descripción personal...</p>
    </div>
    <div class="columna">
      <h2>Datos</h2>
      <ul>
        <li>Ciudad: ...</li>
        <li>Formación: ADSO</li>
      </ul>
    </div>
  </section>

  <section id="proyectos" class="galeria">
    <article class="tarjeta">Proyecto 1</article>
    <article class="tarjeta">Proyecto 2</article>
    <article class="tarjeta">Proyecto 3</article>
    <article class="tarjeta">Proyecto 4</article>
    <article class="tarjeta">Proyecto 5</article>
    <article class="tarjeta">Proyecto 6</article>
```

```
</section>

<footer id="contacto" class="footer">
  <p>© 2026 Mi Nombre</p>
  <div class="redes">
    <a href="#">GitHub</a>
    <a href="#">LinkedIn</a>
  </div>
</footer>

<script src="js/app.js"></script>
</body>
</html>
```


9. Publicación en GitHub paso a paso

La entrega se realiza mediante un repositorio público de GitHub. Siga estos pasos desde la carpeta de su proyecto.

9.1 Inicializar el repositorio local

Abra una terminal en la carpeta mi-portafolio-flex y ejecute:

```
git init
git add .
git commit -m "Estructura inicial del portafolio"
```

9.2 Crear el repositorio en GitHub

6. Ingrese a github.com e inicie sesión.
7. Haga clic en el botón «New» (Nuevo repositorio).
8. Nombre el repositorio: mi-portafolio-flex.
9. Déjelo como público (Public). NO marque la opción de agregar README (ya lo crearemos nosotros).
10. Haga clic en «Create repository».

9.3 Conectar y subir

GitHub mostrará la URL de su repositorio. Use los siguientes comandos (reemplace USUARIO por su nombre de usuario):

```
git remote add origin https://github.com/USUARIO/mi-portafolio-flex.git
git branch -M main
git push -u origin main
```

9.4 Publicar la página con GitHub Pages (opcional pero recomendado)

Para que el portafolio sea visible en línea como una página web real:

11. En el repositorio, vaya a Settings (Configuración).
12. En el menú lateral, seleccione Pages.
13. En «Source», elija la rama main y la carpeta raíz (/root).
14. Guarde. En unos minutos su sitio estará disponible en <https://USUARIO.github.io/mi-portafolio-flex/>

Sobre los commits

No suba todo el proyecto en un único commit final. Realice commits frecuentes y descriptivos a medida que avanza (por ejemplo: «Maqueta el header con flexbox», «Agrega galería

responsive», «Implementa menú hamburguesa»). El historial de commits es parte de la evaluación, porque demuestra un proceso de trabajo ordenado.

10. Criterios de evaluación

Criterio	Peso
Uso correcto y variado de las propiedades del contenedor flexible.	20%
Uso correcto de las propiedades de los ítems (flex, order, align-self).	15%
Todas las secciones obligatorias presentes y maquetadas con Flexbox.	20%
Diseño responsive funcional con al menos dos media queries.	15%
HTML semántico y CSS ordenado, indentado y comentado.	10%
Interactividad con JavaScript (menú o cambio de tema).	10%
Repositorio público en GitHub con commits descriptivos y README.	10%

No se admite el uso de floats ni de frameworks CSS para la maquetación. Detectar su uso implica una penalización en el criterio de propiedades del contenedor.

11. Errores comunes y cómo resolverlos

Síntoma	Causa	Solución
align-content no hace nada	El contenedor es de una sola línea (nowrap).	Agregue flex-wrap: wrap al contenedor.
justify-content no centra vertical	Se confunde el eje principal con el transversal.	Si flex-direction es row, lo vertical se controla con align-items, no con justify-content.
Los ítems no se ajustan a varias líneas	flex-wrap está en su valor por defecto (nowrap).	Defina flex-wrap: wrap o use flex-flow: row wrap.
Un ítem no crece para llenar el espacio	Falta flex-grow o el atajo flex.	Aplique flex: 1 al ítem que debe expandirse.
Margin auto centra mal en columna	El contenedor no tiene altura definida.	Asigne una altura (height o min-height) al contenedor.
order no afecta el lector de pantalla	order es solo visual, no estructural.	Si el orden importa para accesibilidad, reordene en el HTML.
Nada cambia al usar Flexbox	Falta display: flex en el contenedor padre.	Verifique que el padre tenga display: flex.

12. Glosario técnico

Flexbox: módulo de CSS para maquetación flexible y unidireccional de componentes.

Contenedor flexible: elemento padre con display: flex que gobierna a sus hijos directos.

Ítem flexible: cada hijo directo de un contenedor flexible.

Eje principal (main axis): eje a lo largo del cual se distribuyen los ítems; su dirección la define flex-direction.

Eje transversal (cross axis): eje perpendicular al principal.

Responsive: diseño que se adapta automáticamente a distintos tamaños de pantalla.

Media query: regla CSS que aplica estilos según condiciones del dispositivo, como el ancho de la pantalla.

Punto de quiebre (breakpoint): ancho de pantalla en el que el diseño cambia de disposición.

Repositorio: espacio donde se almacena y versiona el código de un proyecto con Git.

Commit: registro de un conjunto de cambios en el historial de Git.

GitHub Pages: servicio gratuito de GitHub para publicar sitios web estáticos.

13. Referencias y recursos

- CSS-Tricks — A Complete Guide to Flexbox — <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- MDN — Conceptos básicos de Flexbox — https://developer.mozilla.org/es/docs/Web/CSS/CSS_flexible_box_layout
- Especificación oficial W3C — CSS Flexible Box Layout Module — <https://www.w3.org/TR/css-flexbox-1/>
- Flexbox Froggy (juego para practicar) — <https://flexboxfroggy.com/#es>
- Documentación de GitHub Pages — <https://docs.github.com/es/pages>

Mensaje final del instructor

Flexbox es una de esas herramientas que, una vez se dominan, cambian para siempre la forma de construir interfaces. Lo que antes tomaba decenas de líneas de CSS confuso, ahora se resuelve en tres o cuatro propiedades claras. Practiquen sin miedo, experimenten con cada valor en las herramientas del navegador, y recuerden que el mejor aprendizaje viene de equivocarse y corregir. Espero ver portafolios creativos y bien maquetados. Mucho éxito.
— Diego Fernando Calderón Silva.

